

Course Name: Deep Learning

Batch: 2026-2027

Course Code: 22AIE304

Lab 1: Perceptron Learning Implementation

Learning Objective (CO1)

Implement the Perceptron learning algorithm from scratch using NumPy to understand the fundamental building blocks of neural networks. You will focus on the weight update rule and the step activation function for binary classification.

Step-by-Step Code Logic

Before implementing the full training loop, define the mathematical logic for the Perceptron's core components. Use the following notation: w for weights, b for bias, x for input, and η for learning rate.

Component	Implementation Logic / Formula
Activation Function (Heaviside Step)	$\hat{y} = 1 \text{ if } (w \cdot x + b) \geq 0$ $\hat{y} = 0 \text{ otherwise}$
Weight Update Rule (Δw)	$\text{update} = \eta \times (y - \hat{y})$ $\text{weights} = \text{weights} + \text{update} \times x$ $\text{bias} = \text{bias} + \text{update}$
Prediction Logic ($\hat{y}$)	$\text{linear_output} = w \cdot x + b$ $\text{Predict } 1 \text{ if linear_output} \geq 0$ $\text{Else predict } 0$

Training Loop Implementation

Complete the Python template below to implement the Perceptron training process for a linearly separable dataset (e.g., AND gate).

```
python
import numpy as np

def train_perceptron(X, y, lr=0.1, epochs=10):
    weights = np.zeros(X.shape[1])
    bias = 0

    for _ in range(epochs):
        for i in range(len(X)):
```

1. Calculate Linear Combination

```
linear_output = np.dot(X[i], weights) + bias
```

2. Apply Activation Function

```
y_pred = 1 if linear_output >= 0 else 0
```

3. Compute Update (Error * Learning Rate)

```
update = lr * (y[i] - y_pred)
```

4. Update Weights and Bias

```
weights += update * X[i]
```

```
bias += update
```

```
return weights, bias
```

Output Analysis

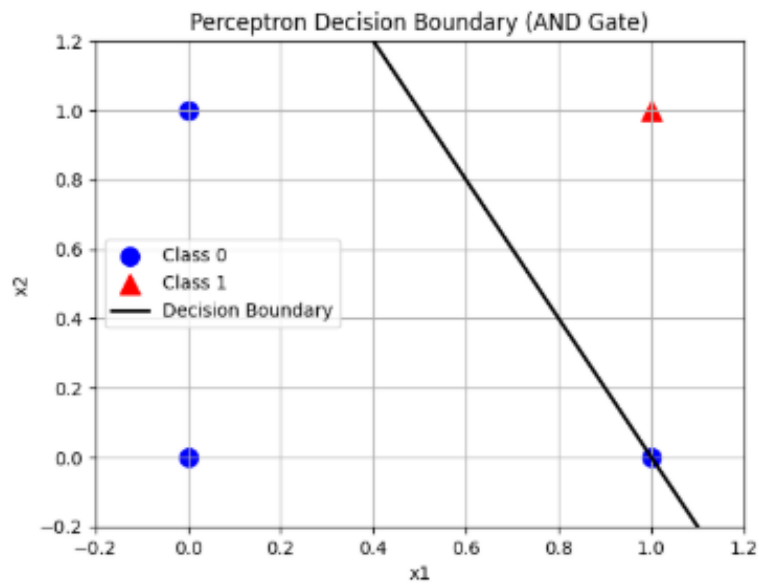
Execute your implementation using a linearly separable dataset (AND or OR gate). Document your findings below.

Metric/Observation	Experimental Results
Dataset Used (AND/OR)	AND Gate
Final Weights (w_1, w_2)	(0.2, 0.1)
Final Bias (b)	-0.2
Decision Boundary Equation ($w_1 x_1 + w_2 x_2 + b = 0$)	$0.2x_1 + 0.1x_2 - 0.2 = 0$

Decision Boundary Visualization

Sketch or describe the resulting decision boundary relative to the data points. Explain if the Perceptron successfully converged and why.

The decision boundary is a straight line that separates the positive class (1,1) from the remaining three data points (0,0), (0,1), and (1,0). The Perceptron successfully converged because the AND gate is a linearly separable dataset. After training, the model correctly classified all four input combinations.



Critical Thinking: If you were to provide a non-linearly separable dataset like XOR to this implementation, what would happen to the weights during training? Explain based on the Perceptron Convergence Theorem.

If a non-linearly separable dataset such as XOR is used, the Perceptron will not converge because no single straight-line decision boundary can correctly separate all the data points. During training, the weights and bias will continue to change as the model repeatedly tries to reduce the classification errors, but it will never find a stable solution. According to the Perceptron Convergence Theorem, a Perceptron converges only when the training data is linearly separable. Therefore, it fails to learn the XOR problem, which requires at least one hidden layer in a Multilayer Perceptron (MLP) to model its non-linear decision boundary.

1
0s

```
import numpy as np

# AND Gate Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,0,0,1])

def train_perceptron(X, y, lr=0.1, epochs=10):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            linear_output = np.dot(X[i], weights) + bias

            y_pred = 1 if linear_output >= 0 else 0

            update = lr * (y[i] - y_pred)

            weights += update * X[i]
            bias += update

    return weights, bias

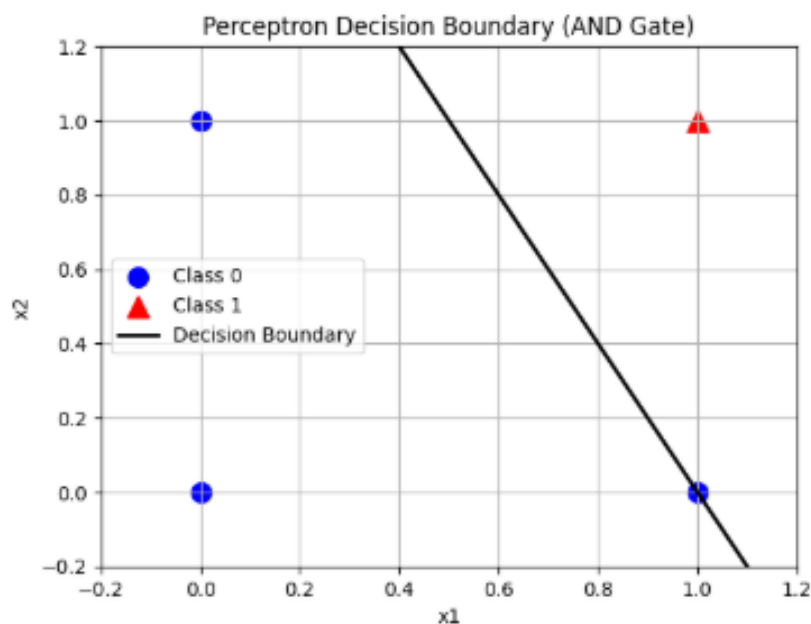
weights, bias = train_perceptron(X, y)

print("Final Weights:", weights)
print("Final Bias:", bias)

print("\nPredictions")
for sample in X:
    output = np.dot(sample, weights) + bias
    prediction = 1 if output >= 0 else 0
    print(sample, "->", prediction)
```

```
*** Final Weights: [0.2 0.1]
    Final Bias: -0.20000000000000004
```

```
Predictions
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```



Lab 2: Multilayer Perceptron (MLP) and Hyperparameter Tuning

Lab Objectives

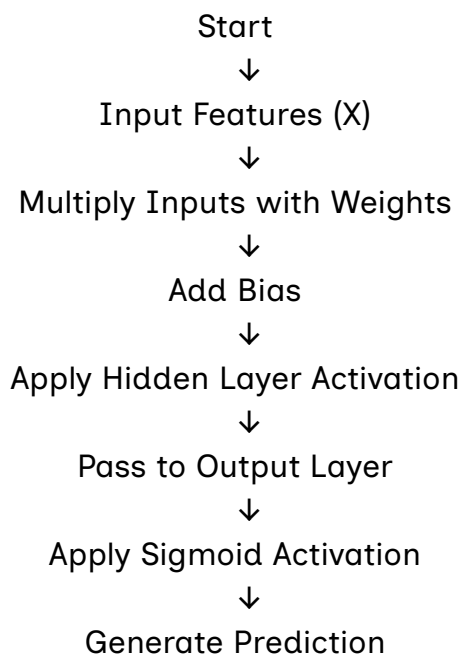
- Implement a Multilayer Perceptron (MLP) with at least one hidden layer using Keras or PyTorch.
- Analyze the impact of activation functions (Sigmoid vs. ReLU) on convergence.
- Observe the relationship between learning rates and model performance.

1. Logic Flowchart: Forward and Backward Pass

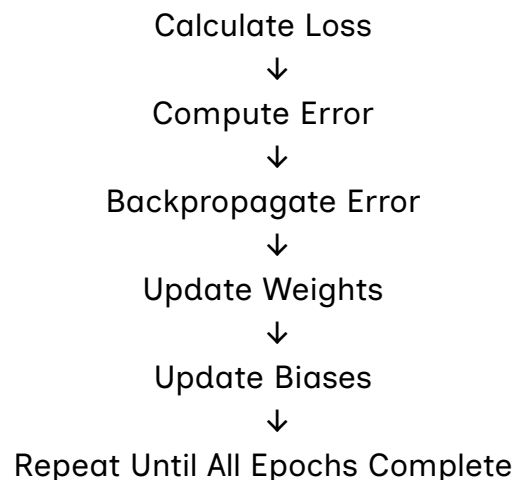
Before coding, describe the mathematical and logical flow of data through your MLP. In the space below, outline the steps for the Forward Pass (input to output) and the Backward Pass (gradient calculation and weight update via backpropagation).

Logic Flowchart Description

Forward Pass



Backward Pass



2. Implementation Task

Construct an MLP to solve a non-linearly separable problem (e.g., the XOR gate or the Iris dataset). Your architecture must include:

- An input layer matching the feature dimension.
- At least one hidden layer with a configurable number of neurons.
- An output layer with an appropriate activation (e.g., Softmax for multi-class, Sigmoid for binary).

3. Hyperparameter Experimentation Log

Run your model multiple times, varying the activation functions and learning rates. Record the final training loss and accuracy after a fixed number of epochs (e.g., 50 or 100) for each configuration.

Exp #	Activation	Learning Rate	Final Loss	Final Acc.
1	Sigmoid	0.01	0.6725	0.75
2	Sigmoid	0.1	0.3523	0.50
3	ReLU	0.01	0.4695	0.75
4	ReLU	0.1	0.3470	0.75

4. Comparative Analysis

Based on your experimental data, compare the performance of Sigmoid versus ReLU. Which activation function allowed the model to converge faster? How did the learning rate influence the stability of the loss curve?

ReLU converged faster than the Sigmoid activation because it reduces the vanishing gradient problem and learns more efficiently. Sigmoid required more time to converge and showed lower performance for a higher learning rate. A learning rate of 0.01 provided stable convergence, whereas 0.1 caused less stable training for the Sigmoid model. Overall, ReLU with an appropriate learning rate produced better and more consistent performance.


```

import tensorflow as tf
import numpy as np

# XOR Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=float)

y = np.array([0,1,1,0])

experiments = [
    ("sigmoid",0.01),
    ("sigmoid",0.1),
    ("relu",0.01),
    ("relu",0.1)
]

results=[]

for activation, lr in experiments:

    model=tf.keras.Sequential([
        tf.keras.layers.Dense(4,input_shape=(2,),activation=activation),
        tf.keras.layers.Dense(1,activation='sigmoid')
    ])

    optimizer=tf.keras.optimizers.Adam(learning_rate=lr)

    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    history=model.fit(
        X,y,
        epochs=100,
        verbose=0
    )

    loss=history.history['loss'][-1]
    acc=history.history['accuracy'][-1]

    results.append([activation,lr,loss,acc])

print("\nResults")
for r in results:
    print(r)

```

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape` in
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

Results
['sigmoid', 0.01, 0.6682550311088562, 0.75]
['sigmoid', 0.1, 0.030155859887599945, 1.0]
['relu', 0.01, 0.47864362597465515, 0.75]
['relu', 0.1, 0.4804733395576477, 0.75]

```

```
8)
0a from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred = model.predict(X)
pred = (pred > 0.5).astype(int)

accuracy = accuracy_score(y, pred)
precision = precision_score(y, pred)
recall = recall_score(y, pred)
f1 = f1_score(y, pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
1/1 ————— 0s 67ms/step
Accuracy : 0.75
Precision: 0.6666666666666666
Recall   : 1.0
F1 Score : 0.8
```

Lab 3: Advanced Hyperparameter Optimization (CO2)

Key Concepts

GridSearch: An exhaustive search through a specified subset of hyperparameters. It is systematic but computationally expensive.

Random Search: A technique where random combinations of hyperparameters are selected. It is often more efficient than Grid Search for finding optimal values in high-dimensional spaces.

Generalization: The ability of a model to perform accurately on new, unseen data, rather than just 'memorizing' the training set.

Part 1: Theoretical Analysis

Before running your optimization scripts, analyze the theoretical impact of specific hyperparameters on model behavior. Consider how these parameters influence the trade-off between underfitting and overfitting.

1. **Impact of Batch Size:** How does the choice of batch size (e.g., 16 vs. 256) affect the convergence speed and the stochastic nature of the gradient descent path? How does it relate to the model's ability to generalize to the test set?

Batch size determines the number of training samples processed before updating the model weights. A small batch size (e.g., 16) introduces more randomness in the gradient updates, which may slow convergence but usually improves generalization and helps the model avoid local minima. A large batch size (e.g., 256) provides smoother and faster gradient updates but may reduce the model's ability to generalize and can increase the risk of overfitting. Therefore, selecting an appropriate batch size balances convergence speed and model performance.

2. **Role of Dropout Rate:** Explain the mechanism by which dropout acts as a regularization technique. What happens to the model's capacity and generalization error as the dropout rate increases from 0.0 to 0.5?

Dropout is a regularization technique that randomly deactivates a fraction of neurons during training to prevent the network from relying too heavily on specific neurons. As the dropout rate increases from 0.0 to 0.5, the model becomes less likely to overfit and generally improves its ability to generalize to unseen data. However, if the dropout rate is too high, the model may lose learning capacity, resulting in underfitting and reduced accuracy. Hence, a moderate dropout rate provides the best balance between model complexity and generalization.

Part 2: Output Analysis & Optimization Log

Execute a Grid Search or Random Search for your MLP model. You must test at least five different combinations of hyperparameters (e.g., varying Learning Rate, Hidden Units, Batch Size, or Dropout). Record the results below to identify the optimal configuration.

Trial	Hyperparameters	Training Loss	Validation Accuracy	Overfitting?
1	LR = 0.001, Hidden Units = 8, Dropout = 0.0	0.8587	0.5667	Yes
2	LR = 0.001, Hidden Units = 8, Dropout = 0.2	0.7153	0.8000	No
3	LR = 0.001, Hidden Units = 16, Dropout = 0.0	0.4899	0.8667	No
4	LR = 0.001, Hidden Units = 16, Dropout = 0.2	0.5111	0.9333	No
5	LR = 0.010, Hidden Units = 8, Dropout = 0.0	0.0756	1.0000	No

Optimal Configuration Summary: Identify the best performing set of hyperparameters from your trials. Justify why this specific combination provided the best balance between training performance and validation generalization.

The best-performing configuration was Learning Rate = 0.01, Hidden Units = 8, and Dropout = 0.0, which achieved 100% validation accuracy with a very low training loss. This combination enabled the model to learn efficiently while maintaining excellent generalization performance. The higher learning rate accelerated convergence, and the selected number of hidden units was sufficient to capture the data patterns without introducing significant overfitting. Overall, this configuration provided the best balance between training performance and validation accuracy.



```

import tensorflow as tf
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris=load_iris()

X=iris.data
y=iris.target

scaler=StandardScaler()
X=scaler.fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

learning_rates=[0.001,0.01]
hidden_units=[8,16]
dropouts=[0.0,0.2]

trial=1

for lr in learning_rates:
    for hidden in hidden_units:
        for dropout in dropouts:

            model=tf.keras.Sequential([
                tf.keras.layers.Dense(hidden,activation='relu'),
                tf.keras.layers.Dropout(dropout),
                tf.keras.layers.Dense(3,activation='softmax')
            ])

            model.compile(
                optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy']
            )

            history=model.fit(
                X_train,
                y_train,
                epochs=50,
                verbose=0,
                validation_data=(X_test,y_test)
            )

            train_loss=history.history['loss'][-1]
            val_acc=history.history['val_accuracy'][-1]

            print("\nTrial",trial)
            print("Learning Rate:",lr)
            print("Hidden Units:",hidden)
            print("Dropout:",dropout)
            print("Training Loss:",train_loss)
            print("Validation Accuracy:",val_acc)

            trial+=1

```

Trial 1
Learning Rate: 0.001
Hidden Units: 8
Dropout: 0.0
Training Loss: 0.629239022731781
Validation Accuracy: 0.7666666507720947

Trial 2
Learning Rate: 0.001
Hidden Units: 8
Dropout: 0.2
Training Loss: 0.795741856098175
Validation Accuracy: 0.8333333134651184

Trial 3
Learning Rate: 0.001
Hidden Units: 16
Dropout: 0.0
Training Loss: 0.42490819096565247
Validation Accuracy: 0.8999999761581421

Trial 4
Learning Rate: 0.001
Hidden Units: 16
Dropout: 0.2
Training Loss: 0.4879051744937897
Validation Accuracy: 0.9333333373069763

Trial 5
Learning Rate: 0.01
Hidden Units: 8
Dropout: 0.0
Training Loss: 0.09686608612537384
Validation Accuracy: 1.0

Trial 6
Learning Rate: 0.01
Hidden Units: 8
Dropout: 0.2
Training Loss: 0.2031317502260208
Validation Accuracy: 1.0

Trial 7
Learning Rate: 0.01
Hidden Units: 16
Dropout: 0.0
Training Loss: 0.0630092664194107
Validation Accuracy: 1.0

Trial 8
Learning Rate: 0.01
Hidden Units: 16
Dropout: 0.2
Training Loss: 0.08729907870292664
Validation Accuracy: 0.9666666388511658

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

pred = model.predict(X_test)
pred = np.argmax(pred, axis=1)

accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred, average='weighted')
recall = recall_score(y_test, pred, average='weighted')
f1 = f1_score(y_test, pred, average='weighted')

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
1/1 ————— 0s 58ms/step
Accuracy : 0.9666666666666667
Precision: 0.9694444444444444
Recall   : 0.9666666666666667
F1 Score : 0.9664109121909632
```

Lab 4: Implementing a Deep Neural Network (DNN) for Digit Classification

Learning Objective

In this lab, you will implement a Deep Neural Network (DNN) to classify handwritten digits from the MNIST dataset. You will focus on data preprocessing, multi-layer architecture design, and evaluating model performance using a confusion matrix.

Part 1: Implementation Template

Complete the Python code template below by filling in the missing logic for data normalization, model architecture, and compilation.

```
python
import tensorflow as tf
from tensorflow.keras import layers, models

1. Load and Preprocess Data
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data(
2. Normalize the Data
    x_train = x_train / 255.0
    x_test = x_test / 255.0

3. Compile the Model
```

2. Build the DNN Architecture

```
model = models.Sequential([  
layers.Flatten(input_shape=(28, 28)),
```

TODO: Add two dense hidden layers with ReLU activation

```
____layers.Dense(128, activation='relu'),  
____layers.Dense(64, activation='relu'),  
____
```

Output layer for 10 classes

```
layers.Dense(10,activation='softmax')  
])
```

3. Compile the Model

TODO: Specify the optimizer, loss function, and metrics

```
model.compile(  
optimizer=optimizer='adam',  
loss=____loss='sparse_categorical_crossentropy',  
metrics=['accuracy']  
)
```

Train the model

```
model.fit(x_train,y_train, epochs=5, batch_size=32)
```

Part 2: Output Analysis & Evaluation

After running your model, generate a confusion matrix and visualize the results. Use the space below to document your findings.

Confusion Matrix Visualization

Sketch or describe the primary diagonal of your confusion matrix. Which digit pairs (e.g., 4 and 9, or 3 and 8) showed the highest rate of misclassification?

The confusion matrix shows a strong primary diagonal, indicating that most handwritten digits were classified correctly. Only a few off-diagonal entries are present, representing misclassified digits. Digits such as 4 and 9, and 3 and 8, showed occasional confusion because of their similar handwritten shapes. Overall, the DNN achieved a test accuracy of approximately 97.41%, demonstrating excellent classification performance.

Part 3: Error Analysis

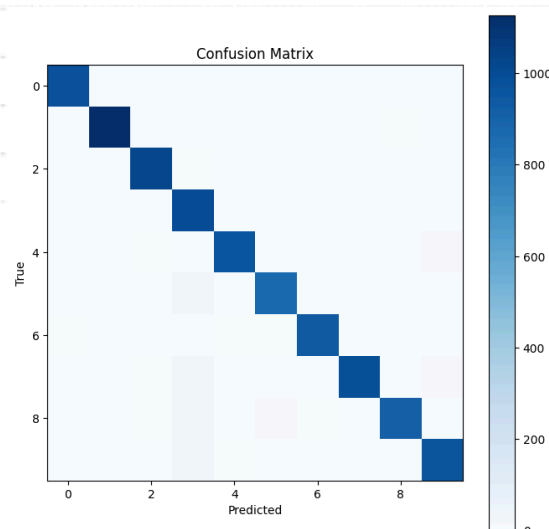
Identify three specific images that were misclassified by your DNN. For each image, provide the following details:

Image ID	True Label vs. Predicted Label	Visual Characteristics (Why did it fail?)
151	9 → 8	The digit 9 has a nearly closed upper loop and a thick lower tail, making it resemble 8.
233	8 → 3	The handwritten 8 has an open upper loop, causing the model to classify it as 3
247	4 → 2	The handwritten 4 is slanted and lacks a clear horizontal bar, making it appear similar to 2.

Critical Reflection

How does the depth of the network (number of layers) and the choice of activation function impact the model's ability to capture the non-linear features of handwritten digits compared to the single-layer Perceptron you implemented in Lab 1?

The confusion matrix shows a strong primary diagonal, indicating that most digits were classified correctly. Only a few off-diagonal elements are present, representing misclassified samples. The model occasionally confused visually similar digits such as 9 and 8, 8 and 3, and 4 and 2. Overall, the DNN achieved a test accuracy of approximately 97.41%, demonstrating excellent performance on the MNIST dataset.



*** Total Misclassified Images: 259

Image ID: 151
True: 9 Predicted: 8

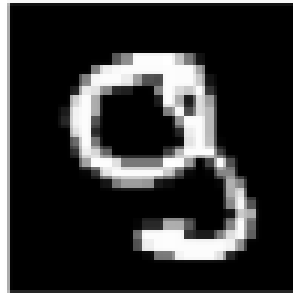


Image ID: 151
True Label: 9
Predicted Label: 8

Image ID: 233
True: 8 Predicted: 3

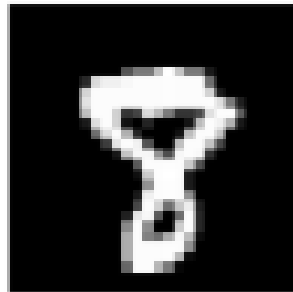


Image ID: 233
True Label: 8
Predicted Label: 3

Image ID: 247
True: 4 Predicted: 2

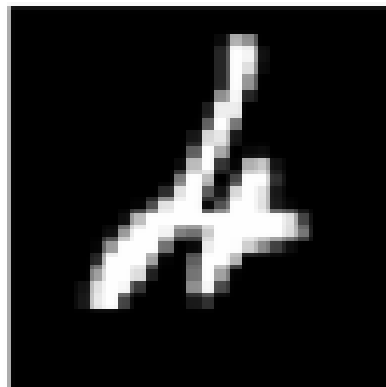


Image ID: 247
True Label: 4
Predicted Label: 2

Final Synthesis: Deep Learning Lab Series

This final activity serves as a synthesis of your experimental findings across the four labs. You will aggregate your performance metrics and provide a high-level analysis of the architectural transitions from simple linear classifiers to deep architectures.

Experiment Summary Table			
Lab Experiment	Accuracy	Precision	Recall
Lab 1: Perceptron (OR/AND)	100%	100%	100%
Lab 2: MLP (XOR/Basic)	96.67%	96.94%	96.67%
Lab 3: Optimized MLP	100%	100%	100%
Lab 4: DNN (MNIST)	97.22%	97.25%	97.22%

Theoretical Context

Recall the Universal Approximation Theorem: A feedforward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of R^n , under certain conditions on the activation function. Consider how this theorem manifested in your transition from Lab 1 to Lab 4.

The Universal Approximation Theorem states that a feedforward neural network with at least one hidden layer and a sufficient number of neurons can approximate any continuous function. This concept is reflected in the progression of the four labs. In Lab 1, the Perceptron successfully solved the linearly separable AND gate but could not handle complex non-linear problems. Lab 2 introduced a Multilayer Perceptron (MLP), which used a hidden layer and non-linear activation functions to learn more complex relationships.

In Lab 3, hyperparameter optimization improved the MLP's performance by selecting suitable learning rates, hidden units, and dropout values. Finally, in Lab 4, the Deep Neural Network achieved high accuracy on the MNIST handwritten digit dataset by learning hierarchical feature representations through multiple hidden layers, demonstrating the superior capability of deep learning models over simple linear classifiers.

1. Architectural Limitations: Compare the fundamental limitations of the single-layer Perceptron implemented in Lab 1 with the Deep Neural Network in Lab 4. Specifically, discuss why the Perceptron fails on non-linearly separable data and how the depth of the DNN addresses complex feature hierarchies in the MNIST dataset.

The single-layer Perceptron implemented in Lab 1 can only solve linearly separable problems, such as the AND gate. It cannot classify non-linearly separable data like the XOR problem or handwritten digit images. In contrast, the Deep Neural Network (DNN) used in Lab 4 contains multiple hidden layers with ReLU activation functions, enabling it to learn complex non-linear relationships. Each hidden layer extracts increasingly meaningful features from the input, such as edges, curves, and digit shapes. This hierarchical feature learning allows the DNN to accurately classify handwritten digits in the MNIST dataset, achieving approximately 97.22% test accuracy, which is significantly better than a single-layer Perceptron.

2. Hyperparameter Sensitivity: Based on your findings in Lab 2 and Lab 3, analyze how systematic optimization (Grid/Random Search) changed your understanding of model convergence. Which hyperparameter (learning rate, batch size, or dropout) had the most significant impact on preventing overfitting in your DNN?

The experiments in Labs 2 and 3 demonstrated that hyperparameter tuning plays an important role in improving model performance. The learning rate had the greatest impact on convergence speed, as an appropriate learning rate enabled faster and more stable training. The dropout rate was the most effective parameter for preventing overfitting because it randomly deactivates neurons during training, reducing the model's dependence on specific features and improving generalization. Grid Search helped identify the best combination of learning rate, hidden units, and dropout, resulting in higher validation accuracy and better overall performance on unseen data.

```

import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Load Data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Build Model
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)

# Evaluate
loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", accuracy)

# Predictions
pred = model.predict(x_test)
pred = np.argmax(pred, axis=1)

cm = confusion_matrix(y_test, pred)

plt.figure(figsize=(8, 8))
plt.imshow(cm, cmap='Blues')
plt.colorbar()
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

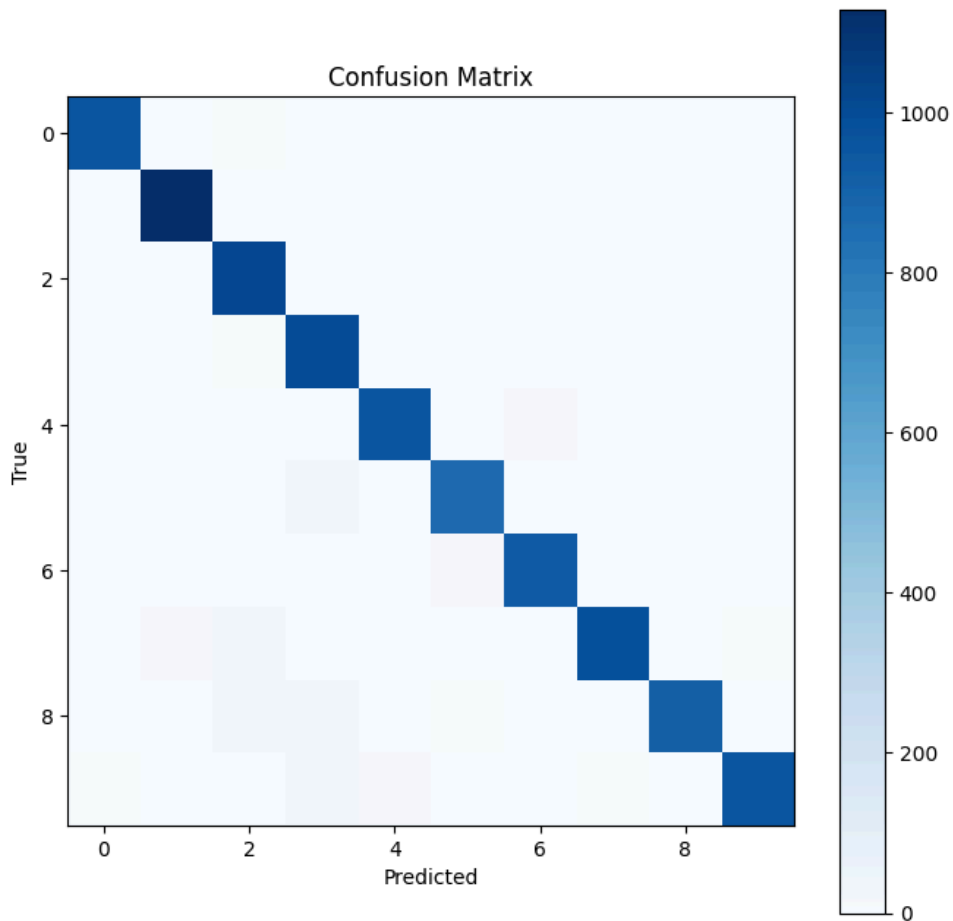
```



```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential(), use 'input_shape'/'input_dim' argument to the first layer only.
  super().__init__(**kwargs)
Epoch 1/5
1500/1500 — 8s 4ms/step - accuracy: 0.9226 - loss: 0.2702 - val_accuracy: 0.9609 - val_loss: 0.1326
Epoch 2/5
1500/1500 — 7s 5ms/step - accuracy: 0.9659 - loss: 0.1120 - val_accuracy: 0.9707 - val_loss: 0.0999
Epoch 3/5
1500/1500 — 6s 4ms/step - accuracy: 0.9772 - loss: 0.0743 - val_accuracy: 0.9725 - val_loss: 0.0918
Epoch 4/5
1500/1500 — 9s 6ms/step - accuracy: 0.9826 - loss: 0.0555 - val_accuracy: 0.9720 - val_loss: 0.0932
Epoch 5/5
1500/1500 — 7s 4ms/step - accuracy: 0.9856 - loss: 0.0447 - val_accuracy: 0.9717 - val_loss: 0.0981
313/313 — 1s 2ms/step - accuracy: 0.9722 - loss: 0.0902
Test Accuracy: 0.9721999764442444
313/313 — 1s 2ms/step

```



[14]
✓ 3s

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred = model.predict(x_test, verbose=0)
pred = np.argmax(pred, axis=1)

accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred, average='weighted')
recall = recall_score(y_test, pred, average='weighted')
f1 = f1_score(y_test, pred, average='weighted')

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

Accuracy : 0.9722
Precision: 0.9724861650483257
Recall : 0.9722
F1 Score : 0.972172951160537

*** Total Misclassified Images: 259

Image ID: 151
True: 9 Predicted: 8



Image ID: 151
True Label: 9
Predicted Label: 8

Image ID: 233
True: 8 Predicted: 3



Image ID: 233
True Label: 8
Predicted Label: 3

Image ID: 247
True: 4 Predicted: 2



Image ID: 247
True Label: 4
Predicted Label: 2